Module 3

UVM Basics

Learning objectives

- Master UVM class hierarchy and phases

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

```
flowchart LR
T1["Introduction to UVM"]
T2["UVM Class Hierarchy"]
T1 --> T2
T3["UVM Phases"]
T2 --> T3
```

Master UVM class hierarchy and phases

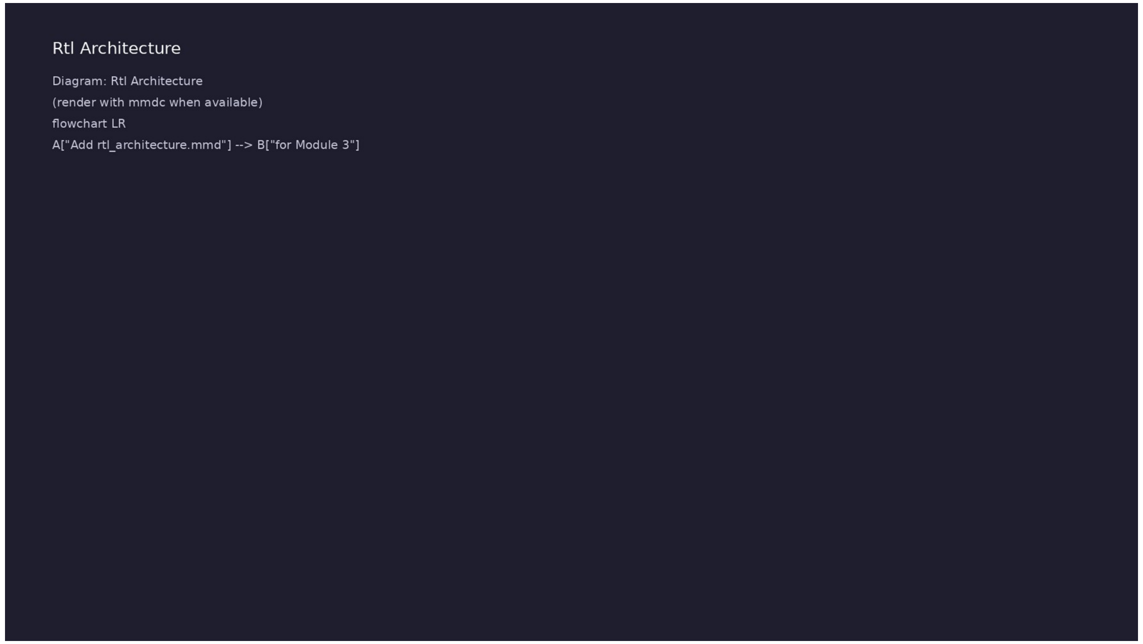
Overview

- This module introduces the Universal Verification Methodology (UVM) using SystemVerilog. You'll lea…

Design architecture

1. UVM class hierarchy (1/3)

- Object hierarchy: `uvm_object` → `uvm_sequence_item` → your transaction
- Component hierarchy: `uvm_component` → driver/monitor/agent/env/test
- Factory macros (`uvm_component_utils`) register types for `type_id::create()`
- Component tree: Test → Env → Agent → {Driver, Monitor, Sequencer}



1. UVM class hierarchy (2/3)

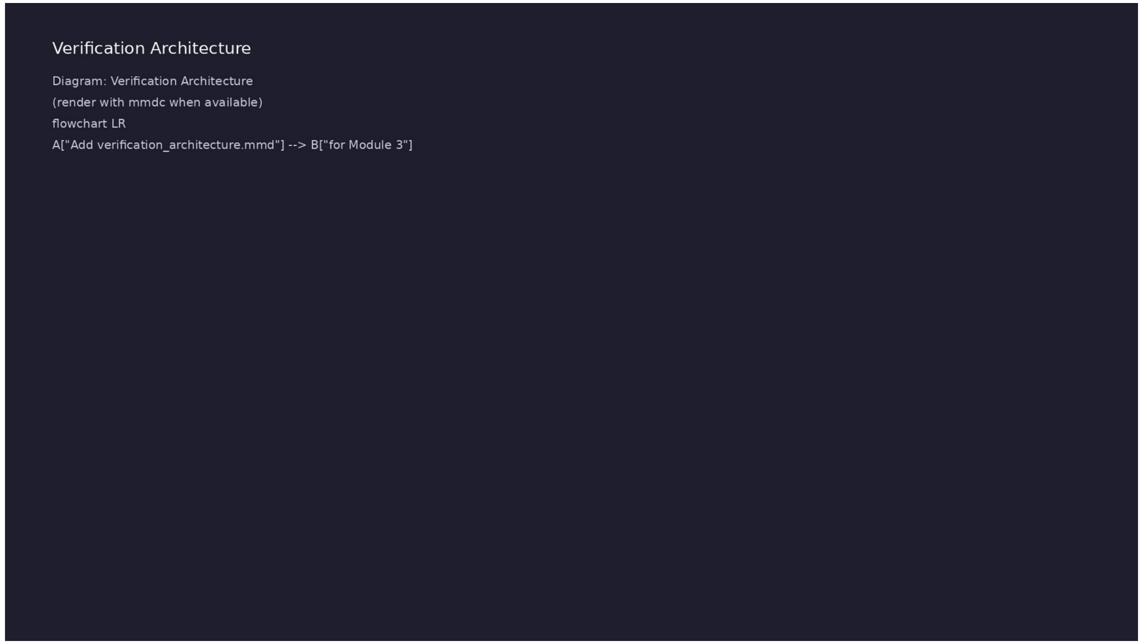
- uvm_test: Top-level; calls run_test(), owns environment
- uvm_env: Contains agents, scoreboard, coverage (as needed)
- uvm_agent: Groups driver, monitor, sequencer on one interface
- uvm_driver: Converts sequence items to pin wiggles

1. UVM class hierarchy (3/3)

- uvm_monitor: Observes bus/DUT; emits transactions
- uvm_sequence_item: Transaction payload (data, address, etc.)

2. UVM phase graph and execution order (1/2)

- Build-time (top-down): `build_phase` → `connect_phase` → `end_of_elaboration_phase`
- Run-time (bottom-up): `run_phase` with objections controlling sim time
- Report: `report_phase` summarizes UVM_INFO/WARNING/ERROR counts
- `super.build_phase(phase)` required in every override to preserve hierarchy

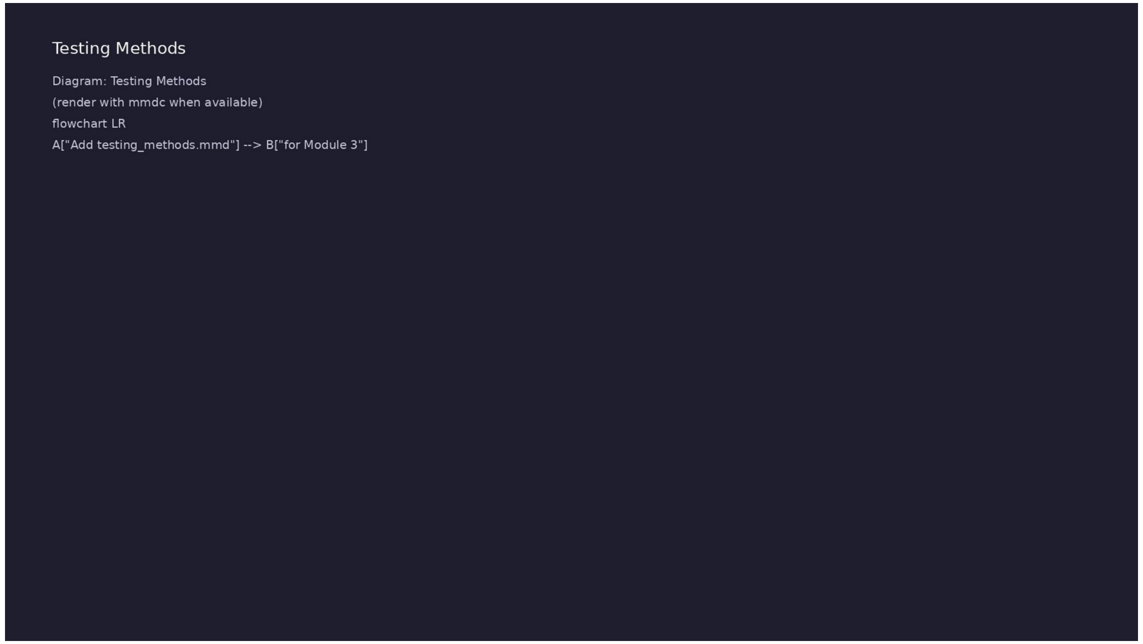


2. UVM phase graph and execution order (2/2)

- Simulation flow: elaboration → `run_test()` → phases → `$finish`

3. Module 3 repository map

- 1: module3/examples/class_hierarchy/ — Minimal full UVM hierarchy demo
- 2: module3/examples/phases/ — Phase callback visibility
- 3: module3/dut/simple_blocks/adder.v — Small RTL for integrated UVM test
- 4: module3/tests/uvm_tests/test_adder_uvm.sv — End-to-end UVM testbench



RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture
(render with mmdc when available)

```
flowchart LR
A["Add rtl_architecture.mmd"] --> B["for Module 3"]
```

Module 3: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
A["Add verification_architecture.mmd"] --> B["for Module 3"]

Module 3: stimulus, observation, and checking.

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module3 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Artifact flow (spec → sim)

- 1: module3/examples/class_hierarchy/ — Minimal full UVM hierarchy demo
- 2: module3/examples/phases/ — Phase callback visibility
- 3: module3/dut/simple_blocks/adder.v — Small RTL for integrated UVM test
- 4: module3/tests/uvm_tests/test_adder_uvm.sv — End-to-end UVM testbench

Example directory layout (1)

- uvm_test: Top-level; calls run_test(), owns environment
- uvm_env: Contains agents, scoreboard, coverage (as needed)
- uvm_agent: Groups driver, monitor, sequencer on one interface
- uvm_driver: Converts sequence items to pin wiggles
- uvm_monitor: Observes bus/DUT; emits transactions

Example directory layout (2)

- uvm_sequence_item: Transaction payload (data, address, etc.)

Directed test execution sequence (1)

- ./scripts/module3.sh --class-hierarchy
- UVM report
- PASS
- elaboration
- run_test()

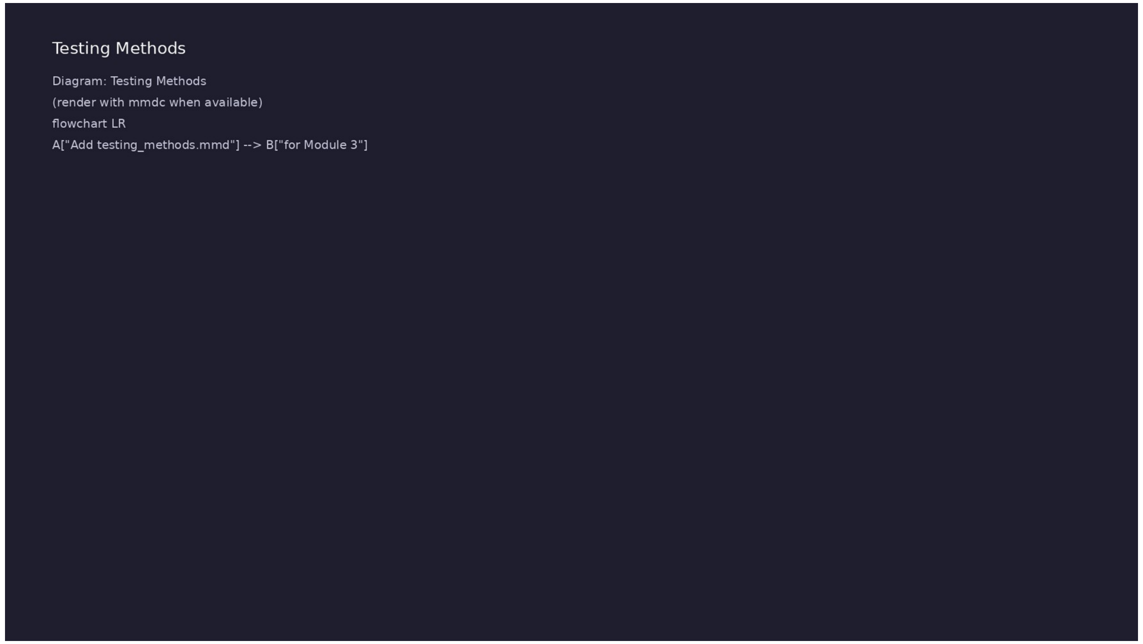
Directed test execution sequence (2)

- phases
- $finish

Verification & testing methods

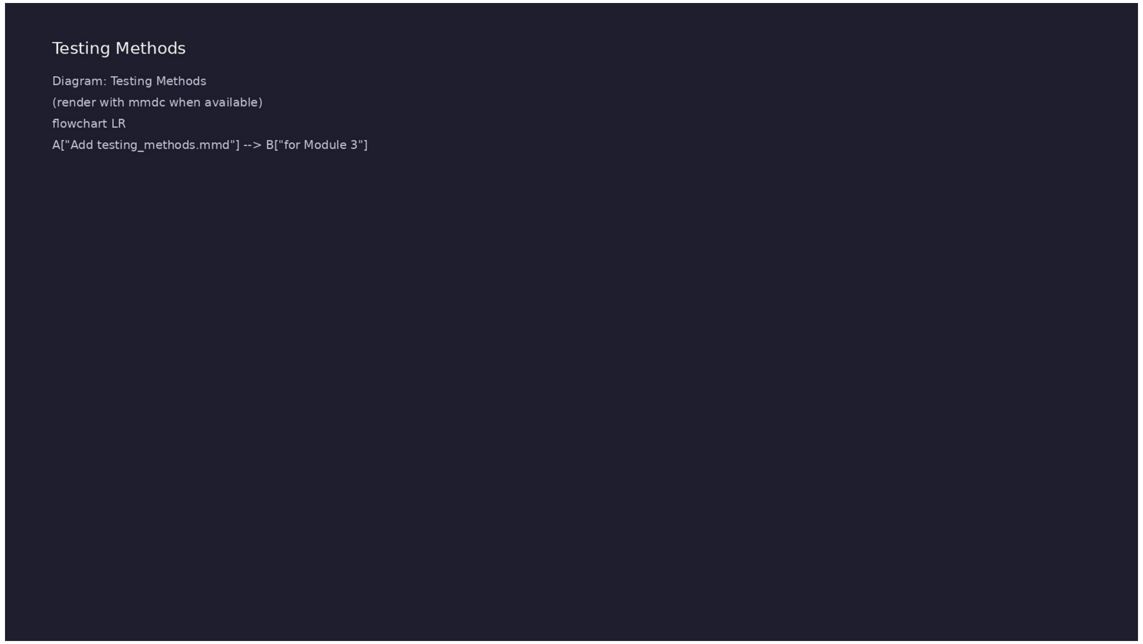
1. UVM infrastructure patterns

- Factory — create components with `type_id::create()`; enable overrides later
- ConfigDB — pass virtual interfaces and config objects down the hierarchy
- Reporting — `uvm_info`/`uvm_error` with verbosity and ID filtering
- Objections — `raise_objection`/`drop_objection` in `run_phase` hold simulation alive



2. Verilator + UVM execution strategy (1/2)

- Compile with `-I$UVM_HOME/src +define+UVM_NO_DPI` (no DPI for Verilator)
- C++ `main` calls `uvm_main` or wrapper that invokes `run_test("MyTest")`
- Start with standalone examples (no DUT) to learn hierarchy and phases
- Progress to `test_adder_uvm` — connect virtual interface to RTL adder



2. Verilator + UVM execution strategy (2/2)

- Closure: zero `UVM_ERROR`/`UVM_FATAL`; expected INFO lines in log

Syllabus topics

Protocol & design details

Detail: Example 3.1: Class Hierarchy (`module3/examples/class_hierarc…

- Transaction Class (`MyTransaction`): Extends `uvm_sequence_item` (uvm_object hierarchy)
- Driver Class (`MyDriver`): Extends `uvm_driver` (uvm_component hierarchy)
- Monitor Class (`MyMonitor`): Extends `uvm_monitor` (uvm_component hierarchy)
- Agent Class (`MyAgent`): Extends `uvm_agent`, demonstrates component composition

Detail: Example 3.1: Class Hierarchy (`module3/examples/class_hierarc…

- Environment Class (`MyEnv`): Extends `uvm_env`, top-level verification environment
- Test Class (`ClassHierarchyTest`): Extends `uvm_test`, orchestrates test execution
- Purpose: Automatically register class with UVM factory
- Usage: Must be inside class definition

Detail: Example 3.2: Phases (`module3/examples/phases/phases.sv`) (1)

- Build-Time Phases: Top-down execution (parent before child)
- Run-Time Phases: Bottom-up execution (child before parent)
- Cleanup Phases: Bottom-up execution (child before parent)
- Phase Execution Order: Demonstrates all UVM phases

Detail: Example 3.2: Phases (`module3/examples/phases/phases.sv`) (2)

- Purpose: Component construction and setup
- Execution: Top-down (parent before child)
- Key: Use `super.build_phase(phase)` to maintain hierarchy
- Purpose: Test execution phases

Detail: Example 3.3: Reporting (`module3/examples/reporting/reporting…

- Severity Levels: INFO, WARNING, ERROR, FATAL
- Verbosity Levels: UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG
- Message Formatting: Formatted messages with data values
- Hierarchical Reporting: Component context in messages

Detail: Example 3.3: Reporting (`module3/examples/reporting/reporting…

- Purpose: Print formatted messages with severity
- Parameters:
- `TAG`: Message category/tag
- `Message`: Text to display

Detail: Example 3.4: ConfigDB (`module3/examples/configdb/configdb.sv…

- Setting Configuration: Global and component-specific
- Getting Configuration: Hierarchical lookup
- Configuration Objects: Complex configuration data
- Scalar Values: Simple configuration values

Detail: Example 3.4: ConfigDB (`module3/examples/configdb/configdb.sv…

- Purpose: Store configuration in database
- Syntax: `uvm_config_db#(type)::set(contxt, path,
  field, value)`
- Parameters:
- `contxt`: Context (usually `this` in test)

Detail: Example 3.5: Factory (`module3/examples/factory/factory.sv`)...

- Factory Registration: Automatic registration via `uvm_object_utils`
- Factory Creation: Using `type_id::create()`
- Type Override: Substituting base class with extended class
- Instance Override: Overriding specific instances

Detail: Example 3.5: Factory (`module3/examples/factory/factory.sv`)…

- Purpose: Automatically register class with factory
- Usage: Must be inside class definition
- Key: Enables factory-based creation and overrides
- Purpose: Create objects/components via factory

Detail: Example 3.6: Objections (`module3/examples/objections/objecti…

- Raising Objections: Keep simulation running
- Dropping Objections: Allow phase completion
- Multiple Objections: Per-component objection tracking
- Coordinated Completion: Phase completes when all objections dropped

Detail: Example 3.6: Objections (`module3/examples/objections/objecti…

- Purpose: Keep phase running until objection is dropped
- Usage: Call at start of `run_phase()` or other run-time phases
- Parameters:
- `this`: Component raising objection

Detail: UVM Factory Macros and Methods (1)

- Purpose: Register class with UVM factory
- Location: Inside class definition
- Enables: Factory creation and overrides
- Syntax: `ClassName::type_id::create(name, parent)`

Detail: UVM Factory Macros and Methods (2)

- Returns: Handle to created object/component
- Key: Factory applies overrides automatically

Detail: UVM Reporting Macros (1)

- `UVM_LOW` - Always displayed (default)
- `UVM_MEDIUM` - Medium detail (most common)
- `UVM_HIGH` - High detail
- `UVM_FULL` - Full detail

Detail: UVM Reporting Macros (2)

- `UVM_DEBUG` - Debug-level detail

Detail: ConfigDB Methods (1)

- Set configuration in test's `build_phase()` before creating components
- Get configuration in component's `build_phase()`
- Always check return value of `get()`
- Provide default values if configuration not found

Detail: ConfigDB Methods (2)

- Use hierarchical paths for component-specific configuration

Detail: Objection Methods

- Must raise objection if doing work in run-time phase
- Must drop objection when work completes
- Phase completes when all objections are dropped
- Each `raise_objection()` must have matching `drop_objection()`

Detail: Component Utility Methods

- Purpose: Component identification
- Usage: Logging, debugging, hierarchical paths

Detail: Sequence Methods (1)

- Purpose: Generate and send transactions to driver
- Usage: Sequences generate test vectors
- Flow: Sequence → Sequencer → Driver → DUT
- Purpose: Post-processing after randomization

Detail: Sequence Methods (2)

- Usage: Calculate expected results, validate data

Detail: TLM Port Methods (Driver) (1)

- Purpose: Driver-sequencer communication
- Methods:
- `get_next_item(txn)`: Get next transaction (blocking)
- `item_done()`: Signal transaction processing complete

Detail: TLM Port Methods (Driver) (2)

- Flow: Sequencer provides → Driver receives → DUT

Detail: Analysis Port Methods (Monitor/Scoreboard) (1)

- Purpose: Broadcast transactions to subscribers
- Usage: Monitor sends transactions to scoreboard, coverage, etc.
- Purpose: Receive transactions from monitor
- Usage: Scoreboard checks DUT behavior

Detail: Analysis Port Methods (Monitor/Scoreboard) (2)

- Purpose: Connect analysis port to analysis export
- Usage: Establish monitor → scoreboard communication

Detail: Virtual Interface Methods

- Purpose: Get virtual interface handle
- Usage: Driver/monitor access DUT signals via virtual interface
- Setting: `uvm_config_db#(virtual interface_type)::set(null, "*", "vif", vif_inst)`

Detail: Top-Level UVM Functions

- Purpose: Start UVM test execution
- Parameters: Test class name (string)
- Behavior: Creates test instance, executes all phases
- Override: `+UVM_TESTNAME=TestName` command line option

Verification flow (reference)

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart LR
A["Add testing_methods.mmd"] --> B["for Module 3"]
```

Stimulus, DUT response, and check points for this module.

1. Introduction to UVM (1/8)

- Universal Verification Methodology
- UVM standard (IEEE 1800.2-2017)
- UVM class library
- Methodology benefits
- Industry adoption
- UVM Benefits

1. Introduction to UVM (2/8)

- Reusability
- Standardization
- Scalability
- Maintainability
- Transaction Class (`MyTransaction`): Extends `uvm_sequence_item` (uvm_object hierarchy)
- Driver Class (`MyDriver`): Extends `uvm_driver` (uvm_component hierarchy)

1. Introduction to UVM (3/8)

- Monitor Class (`MyMonitor`): Extends `uvm_monitor` (uvm_component hierarchy)
- Agent Class (`MyAgent`): Extends `uvm_agent`, demonstrates component composition
- Environment Class (`MyEnv`): Extends `uvm_env`, top-level verification environment
- Test Class (`ClassHierarchyTest`): Extends `uvm_test`, orchestrates test execution
- Purpose: Automatically register class with UVM factory
- Usage: Must be inside class definition

1. Introduction to UVM (4/8)

- Key: Enables factory-based object creation
- Purpose: Initialize UVM objects and components
- Parameters:
- `name`: Component/object name
- `parent`: Parent component (for hierarchy)
- Key: Must call `super.new()` to initialize base class

1. Introduction to UVM (5/8)

- Purpose: Implement UVM phase behavior
- Key: Must call `super.build_phase(phase)` etc. to maintain hierarchy
- Objections: Use in `run_phase()` to control simulation
- Purpose: Create objects/components using factory
- Syntax: `ClassName::type_id::create(name, parent)`
- Key: Factory enables overrides and centralized creation

1. Introduction to UVM (6/8)

- Purpose: Connect component ports in `connect_phase()`
- Usage: Connect driver to sequencer, monitor to scoreboard, etc.
- Purpose: Provide human-readable string representation
- Usage: Debugging, logging, reporting
- `uvm_object` - Base class for all UVM objects (transactions, sequences)
- `uvm_component` - Base class for all UVM components (drivers, monitors, agents)

1. Introduction to UVM (7/8)

- Component hierarchy: Test → Environment → Agent → Driver/Monitor
- Component creation using factory pattern
- Factory macros (`uvm_object_utils`, `uvm_component_utils`) enable registration
- Phase functions/tasks implement component behavior
- Factory creation (`type_id::create()`) enables overrides
- Component hierarchy construction

1. Introduction to UVM (8/8)

- Phase execution demonstration
- Component creation and connection
- UVM report summary

2. UVM Class Hierarchy (1/3)

- Object vs Component
- `uvm_object` - Base for transactions, sequences
- `uvm_component` - Base for testbench components
- Lifetime differences
- Hierarchy differences
- Component Classes

2. UVM Class Hierarchy (2/3)

- `uvm_test` - Top-level test class
- `uvm_env` - Verification environment
- `uvm_agent` - Agent containing driver, monitor, sequencer
- `uvm_driver` - Drives transactions to DUT
- `uvm_monitor` - Observes DUT behavior
- `uvm_sequencer` - Manages sequence execution

2. UVM Class Hierarchy (3/3)

- Object Classes
- `uvm_sequence_item` - Base transaction class
- `uvm_sequence` - Sequence of transactions
- Class relationships and inheritance

3. UVM Phases (1/8)

- Build-Time Phases (Top-down)
- `build_phase()` - Component construction
- `connect_phase()` - Component connections
- `end_of_elaboration_phase()` - Final setup
- `start_of_simulation_phase()` - Simulation start
- Run-Time Phases (Bottom-up)

3. UVM Phases (2/8)

- `pre_reset_phase()`, `reset_phase()`, `post_reset_phase()`
- `pre_configure_phase()`, `configure_phase()`, `post_configure_phase()`
- `pre_main_phase()`, `main_phase()`, `post_main_phase()`
- `pre_shutdown_phase()`, `shutdown_phase()`, `post_shutdown_phase()`
- Cleanup Phases (Bottom-up)
- `extract_phase()` - Extract results

3. UVM Phases (3/8)

- `check_phase()` - Final checks
- `report_phase()` - Generate reports
- `final_phase()` - Final cleanup
- Build-Time Phases: Top-down execution (parent before child)
- Run-Time Phases: Bottom-up execution (child before parent)
- Cleanup Phases: Bottom-up execution (child before parent)

3. UVM Phases (4/8)

- Phase Execution Order: Demonstrates all UVM phases
- Purpose: Component construction and setup
- Execution: Top-down (parent before child)
- Key: Use `super.build_phase(phase)` to maintain hierarchy
- Purpose: Test execution phases
- Execution: Bottom-up (child before parent)

3. UVM Phases (5/8)

- Key: Use objections in `main_phase()` to control simulation
- Purpose: Result extraction and reporting
- Execution: Bottom-up (child before parent)
- Key: Use for final verification and reporting
- Purpose: Get component identification
- Usage: Logging, debugging, hierarchical paths

3. UVM Phases (6/8)

- Build-time phases execute top-down (parent before child)
- Run-time phases execute bottom-up (child before parent)
- Cleanup phases execute bottom-up (child before parent)
- Phases enable structured testbench execution
- Build-time phases are functions (no delays allowed)
- Run-time phases are tasks (can have delays)

3. UVM Phases (7/8)

- Cleanup phases are functions (no delays allowed)
- Must call `super.phase_name(phase)` to maintain hierarchy
- Use `get_name()` and `get_full_name()` for component identification
- All build-time phases executed in order
- All run-time phases executed in order
- All cleanup phases executed in order

3. UVM Phases (8/8)

- Phase execution logging

4. UVM Reporting (1/8)

- UVM Messaging System
- Severity levels: FATAL, ERROR, WARNING, INFO, DEBUG
- Verbosity levels: UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG
- Message formatting
- Hierarchical reporting
- Severity Levels: INFO, WARNING, ERROR, FATAL

4. UVM Reporting (2/8)

- Verbosity Levels: UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG
- Message Formatting: Formatted messages with data values
- Hierarchical Reporting: Component context in messages
- Purpose: Print formatted messages with severity
- Parameters:
- `TAG`: Message category/tag

4. UVM Reporting (3/8)

- `Message`: Text to display
- `Verbosity`: For `uvm_info` only (UVM_LOW, UVM_MEDIUM, etc.)
- Usage: Standard UVM messaging
- Purpose: Include data values in messages
- Uses: `$sformatf()` for string formatting
- Key: Combine UVM macros with SystemVerilog formatting

4. UVM Reporting (4/8)

- Purpose: Control message visibility
- Usage: Set in `build_phase()` or test
- Levels: UVM_LOW < UVM_MEDIUM < UVM_HIGH < UVM_FULL < UVM_DEBUG
- Purpose: Include component hierarchy in messages
- Uses: `get_full_name()` for hierarchical path
- INFO: Informational messages (non-critical)

4. UVM Reporting (5/8)

- WARNING: Potential issues (non-fatal)
- ERROR: Actual errors (may stop simulation)
- FATAL: Fatal errors (stops simulation immediately)
- UVM_LOW: Always displayed (default)
- UVM_MEDIUM: Medium detail (default for most messages)
- UVM_HIGH: High detail

4. UVM Reporting (6/8)

- UVM_FULL: Full detail
- UVM_DEBUG: Debug-level detail
- Severity indicates message importance
- Verbosity controls message visibility
- Messages include component hierarchy context
- Reporting enables debugging and monitoring

4. UVM Reporting (7/8)

- `uvm_info` is most common (with verbosity control)
- `uvm_warning` for non-critical issues
- `uvm_error` for errors (may stop simulation)
- `uvm_fatal` for fatal errors (stops simulation)
- Use `$sformatf()` for formatted messages with data
- Different severity levels demonstrated

4. UVM Reporting (8/8)

- Verbosity control demonstrated
- Formatted messages with data
- Hierarchical message context

5. ConfigDB (1/7)

- Configuration Database Basics
- Setting configuration
- Getting configuration
- Configuration hierarchy
- Configuration objects
- Setting Configuration: Global and component-specific

5. ConfigDB (2/7)

- Getting Configuration: Hierarchical lookup
- Configuration Objects: Complex configuration data
- Scalar Values: Simple configuration values
- Purpose: Store configuration in database
- Syntax: `uvm_config_db#(type)::set(contxt, path, field, value)`
- Parameters:

5. ConfigDB (3/7)

- `contxt`: Context (usually `this` in test)
- `path`: Component path (`"*"` for global, `"comp_name"` for specific)
- `field`: Configuration field name (string)
- `value`: Value to store
- Usage: Set in test's `build_phase()` before creating components
- Purpose: Retrieve configuration from database

5. ConfigDB (4/7)

- Syntax: `uvm_config_db#(type)::get(contxt, path, field, var)`
- Returns: `1` if found, `0` if not found
- Usage: Get in component's `build_phase()`
- Key: Always check return value, provide defaults if not found
- Purpose: String representation of configuration
- Usage: Debugging, logging

5. ConfigDB (5/7)

- ConfigDB enables flexible test configuration
- Configuration can be set globally or per-component
- Components look up configuration in `build_phase()`
- Hierarchical paths enable component-specific config
- Set before create: Configuration must be set before components are created
- Get in build_phase: Components get configuration in their `build_phase()`

5. ConfigDB (6/7)

- Check return value: Always check if `get()` succeeded
- Provide defaults: Use default values if configuration not found
- Type safety: ConfigDB is type-safe (uses template parameter)
- Configuration setting and getting
- Hierarchical configuration lookup
- Component-specific configuration

5. ConfigDB (7/7)

- Configuration object usage

6. Factory Pattern (1/6)

- Factory Pattern Basics
- Factory registration
- Factory creation
- Type override
- Instance override
- Factory Registration: Automatic registration via `` `uvm_object_utils` ``

6. Factory Pattern (2/6)

- Factory Creation: Using `type_id::create()`
- Type Override: Substituting base class with extended class
- Instance Override: Overriding specific instances
- Purpose: Automatically register class with factory
- Usage: Must be inside class definition
- Key: Enables factory-based creation and overrides

6. Factory Pattern (3/6)

- Purpose: Create objects/components via factory
- Syntax: `ClassName::type_id::create(name, parent)`
- Key: Factory applies overrides automatically
- Purpose: Replace all instances of base class with extended class
- Usage: Set in test's `build_phase()` before creating components
- Effect: All `BaseDriver::type_id::create()` calls create `ExtendedDriver`

6. Factory Pattern (4/6)

- Purpose: Replace specific instance with extended class
- Usage: Set in test's `build_phase()` before creating components
- Effect: Only specified instance is overridden
- Purpose: Get type wrapper for override operations
- Usage: Used in override methods
- Factory enables centralized object creation

6. Factory Pattern (5/6)

- Type override affects all instances
- Instance override affects specific instances
- Overrides enable test customization
- Factory macros (`uvm_object_utils`, `uvm_component_utils`) register classes
- Factory creation (`type_id::create()`) applies overrides automatically
- Set overrides before create: Overrides must be set before creating components

6. Factory Pattern (6/6)

- Type override: Affects all instances of the base class
- Instance override: Affects only the specified instance
- Factory creation demonstrated
- Type override demonstrated
- Instance override demonstrated
- Override effects verified

7. Objections (1/6)

- Objection Mechanism
- Raising objections
- Dropping objections
- Phase completion
- Simulation control
- Raising Objections: Keep simulation running

7. Objections (2/6)

- Dropping Objections: Allow phase completion
- Multiple Objections: Per-component objection tracking
- Coordinated Completion: Phase completes when all objections dropped
- Purpose: Keep phase running until objection is dropped
- Usage: Call at start of `run_phase()` or other run-time phases
- Parameters:

7. Objections (3/6)

- `this`: Component raising objection
- `description`: Optional description for named objections
- Key: Must raise objection if doing work in run-time phase
- Purpose: Signal that work is complete
- Usage: Call when work in phase is finished
- Key: Must drop objection when work completes

7. Objections (4/6)

- Pattern: Raise at start, drop when complete
- Key: Ensures phase doesn't end prematurely
- Purpose: Track multiple independent tasks
- Usage: Use named objections for different tasks
- Key: Each named objection is tracked separately
- Raise objection to keep phase running

7. Objections (5/6)

- Drop objection when work completes
- Phase completes when all objections dropped
- Multiple objections per component supported
- Raise at start: Call `raise_objection()` at beginning of work
- Drop when done: Call `drop_objection()` when work completes
- Phase completion: Phase ends when all objections are dropped

7. Objections (6/6)

- Named objections: Use for tracking multiple tasks
- Must match: Each `raise_objection()` must have matching `drop_objection()`
- Objection raising and dropping
- Phase completion control
- Multiple objection handling
- Coordinated phase completion

8. UVM Functions and Methods Reference (1/8)

- Purpose: Register class with UVM factory
- Location: Inside class definition
- Enables: Factory creation and overrides
- Syntax: `ClassName::type_id::create(name, parent)`
- Returns: Handle to created object/component
- Key: Factory applies overrides automatically

8. UVM Functions and Methods Reference (2/8)

- `UVM_LOW` - Always displayed (default)
- `UVM_MEDIUM` - Medium detail (most common)
- `UVM_HIGH` - High detail
- `UVM_FULL` - Full detail
- `UVM_DEBUG` - Debug-level detail
- Set configuration in test's `build_phase()` before creating components

8. UVM Functions and Methods Reference (3/8)

- Get configuration in component's `build_phase()`
- Always check return value of `get()`
- Provide default values if configuration not found
- Use hierarchical paths for component-specific configuration
- Must raise objection if doing work in run-time phase
- Must drop objection when work completes

8. UVM Functions and Methods Reference (4/8)

- Phase completes when all objections are dropped
- Each `raise_objection()` must have matching `drop_objection()`
- Purpose: Component identification
- Usage: Logging, debugging, hierarchical paths
- Purpose: Generate and send transactions to driver
- Usage: Sequences generate test vectors

8. UVM Functions and Methods Reference (5/8)

- Flow: Sequence → Sequencer → Driver → DUT
- Purpose: Post-processing after randomization
- Usage: Calculate expected results, validate data
- Purpose: Driver-sequencer communication
- Methods:
- `get_next_item(txn)`: Get next transaction (blocking)

8. UVM Functions and Methods Reference (6/8)

- \`item_done()\`: Signal transaction processing complete
- Flow: Sequencer provides → Driver receives → DUT
- Purpose: Broadcast transactions to subscribers
- Usage: Monitor sends transactions to scoreboard, coverage, etc.
- Purpose: Receive transactions from monitor
- Usage: Scoreboard checks DUT behavior

8. UVM Functions and Methods Reference (7/8)

- Purpose: Connect analysis port to analysis export
- Usage: Establish monitor → scoreboard communication
- Purpose: Get virtual interface handle
- Usage: Driver/monitor access DUT signals via virtual interface
- Setting: `uvm_config_db#(virtual interface_type)::set(null, "*", "vif", vif_inst)`
- Purpose: Start UVM test execution

8. UVM Functions and Methods Reference (8/8)

- Parameters: Test class name (string)
- Behavior: Creates test instance, executes all phases
- Override: `+UVM_TESTNAME=TestName` command line option

Run: Example 3.1: Class Hierarchy

```
./scripts/module3.sh --class-hierarchy
cd module3/examples/class_hierarchy
verilator -sv --cc --exe --timing --trace \
-I$UVM_HOME/src +incdir+$UVM_HOME/src +define+UVM_NO_DPI \
--binary $UVM_HOME/src/uvm_pkg.sv class_hierarchy.sv
class_hierarchy.cpp
\
--top-module class_hierarchy
make -C obj_dir -f Vclass_hierarchy.mk
./obj_dir/Vclass_hierarchy
```

Run: Example 3.2: Phases

```
./scripts/module3.sh --phases
```

Run: Example 3.3: Reporting

```
./scripts/module3.sh --reporting
```

Run: Example 3.4: ConfigDB

```
./scripts/module3.sh --configdb
```

Run: Example 3.5: Factory

```
./scripts/module3.sh --factory
```

Run: Example 3.6: Objections

```
./scripts/module3.sh --objections
```

Hands-on examples

Module 3 self-check

```
$ ./scripts/module3.sh --help
```

Terminal: module3_check

```
Usage: ./scripts/module3.sh [OPTIONS]

Module 3: UVM Basics
This script runs examples and tests for Module 3.

OPTIONS:
  UVM Examples:
    --class-hierarchy      Run class hierarchy examples
    --phases               Run phases examples
    --reporting            Run reporting examples
    --configdb             Run ConfigDB examples
    --factory              Run factory pattern examples
    --objections           Run objections examples
    --all-examples         Run all UVM examples (default)
    --skip-examples        Skip all UVM examples

  Tests:
    --uvm-tests            Run UVM testbenches
    --all-tests            Run all tests

  Environment:
    --sim SIMULATOR        Simulator to use (default: verilator)
    --uvm-home DIR         UVM library directory (default: $UVM_HOME)
    --jobs N               Number of parallel build jobs (default: 8)
    --no-clean             Skip cleaning before build (faster rebuilds)

  Other:
    --help, -h             Show this help message

EXAMPLES:
  # Run all UVM examples (with parallel builds)
  ./scripts/module3.sh

  # Run only class hierarchy
  ./scripts/module3.sh --class-hierarchy
```

Exercise scaffold

```
./scripts/module3.sh --scaffold
```

Example 1: Class Hierarchy

- Transaction Class (`MyTransaction`): Extends `uvm_sequence_item` (uvm_object hierarchy)
- Driver Class (`MyDriver`): Extends `uvm_driver` (uvm_component hierarchy)
- Monitor Class (`MyMonitor`): Extends `uvm_monitor` (uvm_component hierarchy)
- Agent Class (`MyAgent`): Extends `uvm_agent`, demonstrates component composition
- Environment Class (`MyEnv`): Extends `uvm_env`, top-level verification environment

Demo: Class Hierarchy

```
$ ./scripts/module3.sh --class-hierarchy
```

Terminal: ex01_class_hierarchy

```
□[0;36m========================================□[0m
□[0;36mModule 3: UVM Basics□[0m
□[0;36m========================================□[0m

□[0;34m[2026-05-31 03:05:12] Log file: /home/yongfu/proj/universal-verification-methodology/lear
□[0;34m[2026-05-31 03:05:12] Checking prerequisites...□[0m
□[0;31m[2026-05-31 03:05:12] Error: verilator not found. Please install it first.□[0m
```

Example 2: Phases

- Build-Time Phases: Top-down execution (parent before child)
- Run-Time Phases: Bottom-up execution (child before parent)
- Cleanup Phases: Bottom-up execution (child before parent)
- Phase Execution Order: Demonstrates all UVM phases

Demo: Phases

```
$ ./scripts/module3.sh --phases
```

Terminal: ex02_phases

```
[0;36m========================================[0m
[0;36mModule 3: UVM Basics[0m
[0;36m========================================[0m

[0;34m[2026-05-31 03:05:13] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0;34m[2026-05-31 03:05:13] Checking prerequisites...[0m
[0;31m[2026-05-31 03:05:13] Error: verilator not found. Please install it first.[0m
```

Example 3: Reporting

- Severity Levels: INFO, WARNING, ERROR, FATAL
- Verbosity Levels: UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL, UVM_DEBUG
- Message Formatting: Formatted messages with data values
- Hierarchical Reporting: Component context in messages

Demo: Reporting

```
$ ./scripts/module3.sh --reporting
```

Terminal: ex03_reporting

```
[0;36m========================================[0m
[0;36mModule 3: UVM Basics[0m
[0;36m========================================[0m

[0;34m[2026-05-31 03:05:13] Log file: /home/yongfu/proj/universal-verification-methodology/lear
[0;34m[2026-05-31 03:05:13] Checking prerequisites...[0m
[0;31m[2026-05-31 03:05:13] Error: verilator not found. Please install it first.[0m
```

Example 4: ConfigDB

- Setting Configuration: Global and component-specific
- Getting Configuration: Hierarchical lookup
- Configuration Objects: Complex configuration data
- Scalar Values: Simple configuration values

Demo: ConfigDB

```
$ ./scripts/module3.sh --configdb
```

Terminal: ex04_configdb

```
□[0;36m========================================□[0m
□[0;36mModule 3: UVM Basics□[0m
□[0;36m========================================□[0m

□[0;34m[2026-05-31 03:05:14] Log file: /home/yongfu/proj/universal-verification-methodology/lear
□[0;34m[2026-05-31 03:05:14] Checking prerequisites...□[0m
□[0;31m[2026-05-31 03:05:14] Error: verilator not found. Please install it first.□[0m
```

Example 5: Factory

- Factory Registration: Automatic registration via `uvm_object_utils`
- Factory Creation: Using `type_id::create()`
- Type Override: Substituting base class with extended class
- Instance Override: Overriding specific instances

Demo: Factory

```
$ ./scripts/module3.sh --factory
```

Terminal: ex05_factory

```
□[0;36m========================================□[0m
□[0;36mModule 3: UVM Basics□[0m
□[0;36m========================================□[0m

□[0;34m[2026-05-31 03:05:14] Log file: /home/yongfu/proj/universal-verification-methodology/lear
□[0;34m[2026-05-31 03:05:14] Checking prerequisites...□[0m
□[0;31m[2026-05-31 03:05:15] Error: verilator not found. Please install it first.□[0m
```

Example 6: Objections

- Raising Objections: Keep simulation running
- Dropping Objections: Allow phase completion
- Multiple Objections: Per-component objection tracking
- Coordinated Completion: Phase completes when all objections dropped

Demo: Objections

```
$ ./scripts/module3.sh --objections
```

Terminal: ex06_objections

```
□[0;36m========================================□[0m
□[0;36mModule 3: UVM Basics□[0m
□[0;36m========================================□[0m

□[0;34m[2026-05-31 03:05:15] Log file: /home/yongfu/proj/universal-verification-methodology/lear
□[0;34m[2026-05-31 03:05:15] Checking prerequisites...□[0m
□[0;31m[2026-05-31 03:05:15] Error: verilator not found. Please install it first.□[0m
```

Common pitfalls

Class Hierarchy

- Create transaction classes extending uvm_sequence_item
- Create component classes extending uvm_component
- Build component hierarchy
- Test component creation

Phases

- Implement all build-time phases
- Implement run-time phases
- Implement cleanup phases
- Verify phase execution order

Reporting

- Use different severity levels
- Control verbosity
- Format messages with data
- Test hierarchical reporting

ConfigDB

- Set global configuration
- Set component-specific configuration
- Get configuration in components
- Use configuration objects

Factory

- Register classes with factory
- Create objects using factory
- Implement type overrides
- Implement instance overrides

Objections

- Raise objections in run_phase
- Drop objections when work completes
- Test multiple objections
- Verify phase completion

Practice & assessment

What you should know (1/2)

- Understand UVM class hierarchy (uvm_object vs uvm_component)
- Master UVM phases (build-time, run-time, cleanup)
- Use UVM reporting effectively (severity, verbosity, formatting)
- Configure components using ConfigDB
- Understand factory pattern (registration, creation, overrides)
- Control simulation with objections

What you should know (2/2)

- Structure UVM testbenches properly

Exercises

- Class Hierarchy
- Phases
- Reporting
- ConfigDB
- Factory

Exercises

- Objections

Assessment checklist

- Understands UVM class hierarchy
- Masters UVM phases
- Uses UVM reporting effectively
- Configures components using ConfigDB
- Understands factory pattern
- Controls simulation with objections

Summary & next steps

- Master UVM class hierarchy and phases
- Complete module3/CHECKLIST.md
- Review module3/EXAMPLES.md and run each lab
- Next: Next module in course